Projeto IC - Meta 2

Otimização de Híper-parâmetros com Swarm Intelligence

Ana Rita Conceição Pessoa - 2023112690

David José Gomes de Matos - 2023143725

Coimbra, novembro de 2025

Objetivos do Trabalho

- Perceber como funcionam os algoritmos Swarm (GSA e PSO)
- Testar os algoritmos na função de benchmark de Ackley
- Usar Swarm para otimizar os hiper-parâmetros de uma CNN
- Comparar o desempenho dos algoritmos

O que é Computação Swarm?

Computação Swarm é um paradigma da inteligência computacional inspirado em comportamentos coletivos observados na natureza

Baseia-se na **interação local** de múltiplos agentes simples que, através da cooperação, procuram soluções para problemas complexos

Processo **descentralizado e robusto** onde comportamentos inteligentes emergem da interação do grupo



Existem vários agentes simples a cooperar
No conjunto, o comportamento inteligente surge

Porquê usar Swarm na otimização?

Evita a procura exaustiva

Os algoritmos Swarm exploram o espaço de procura de forma mais eficiente, evitando a necessidade de avaliar todas as combinações de parâmetros, tornando-se inviável em espaços de alta dimensão.

Funciona bem em superfícies com muitos mínimos locais

A exploração paralela permite que o algoritmo escape de mínimos locais e encontre soluções mais próximas do ótimo global em funções objetivo complexas.

Boa escolha para ajustar hiper-parâmetros

Em contextos como o treino de redes neuronais, onde o espaço de hiper-parâmetros é não linear e ruidoso, os algoritmos Swarm fornecem uma alternativa robusta para encontrar configurações ótimas.

Diferenças GSA vs PSO

EXPLORAÇÃO

GSA Gravitational Search Algorithm

Movimento mais disperso → **exploração ampla**

- Baseia-se em **metáfora física** (gravidade entre agentes)
- Agentes com melhor fitness têm **maior massa** e atraem mais os outros
- **Exploração inicial forte** → movimento mais caótico no início
- Desempenho melhora com o **número de iterações**

VS

CONVERGÊNCIA

PSO Particle Swarm Optimization

Movimento direcionado → **convergência rápida**

- Baseia-se em partículas que **atualizam velocidade**
- Partículas com **memória própria** (pbest) e seguem a melhor do grupo (gbest)
- **Convergência mais suave** e estável
- Muito sensível ao **parâmetro de inércia w**

Algoritmos inspiados na natureza com diferentes **comportamentos de procura e convergência**

Como funciona o GSA

Cálculo das Massas

A massa de cada agente é determinada pelo seu valor de fitness, onde um fitness menor corresponde a uma massa maior.

Cálculo das Forças

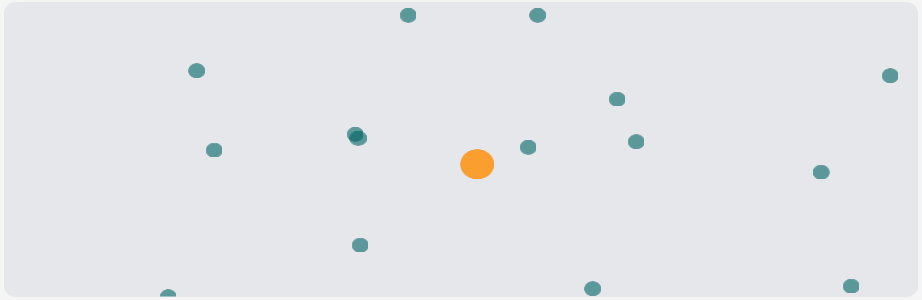
As forças gravitacionais são calculadas entre pares de agentes, sendo a magnitude proporcional às massas e inversamente proporcional à distância.

Atualização de Aceleração/velocidade

A força resultante sobre cada agente é usada para calcular a sua aceleração, que atualiza a velocidade.

Atualização de Posições

A nova velocidade é aplicada para atualizar a posição do agente no espaço de procura.



```python
# Loop principal do GSA
for t in range(iteration):

    # Factor aleatório (controlo estocástico, como ruído no movimento)
    csi = np.random.random((n, dimension))
    eps = np.random.random((1, n))[0]

    # Avaliação do fitness de cada agente
    fitness = np.array([function(x) for x in self.__agents])

    # Cálculo das massas normalizadas:
    # agentes com menor fitness (melhores soluções) têm maior massa
    m = np.array([(function(x) - max(fitness)) /
                  (min(fitness) - max(fitness)) for x in self.__agents])
    M = np.array([i / sum(m) for i in m])   # normalização

    # Actualização da constante gravitacional (decresce com o tempo)
    G = G0 / exp(0.01 * t)

    # Cálculo da força e aceleração resultante em cada agente
    # Soma das forças exercidas pelos restantes agentes
    a = np.array([sum([eps[j] * G * M[j] *
                       (self.__agents[j] - self.__agents[i]) / (
                    np.linalg.norm(self.__agents[i] - self.__agents[j]) + 0.001)
                       for j in range(n)]) for i in range(n)])

    # Actualização da velocidade (com termo aleatório 'csi')
    velocity = csi * velocity + np.array([a[i] for i in range(n)])

    # Actualização da posição
    self.__agents += velocity
    # Garante que as posições não saem dos limites definidos
    self.__agents = np.clip(self.__agents, lb, ub)
```

</> Excerto de código de **gsa.py**

Como funciona o PSO

Parâmetros do PSO

O PSO utiliza conceitos de velocidade para guiar as partículas.
A velocidade de cada partícula é influenciada pela sua melhor posição individual (**p_best**) e pela melhor posição global encontrada por todo o enxame (**g_best**).

Efeito dos Parâmetros

- **w** (fator de inércia): Controla a influência da velocidade anterior da partícula
- **c1** (peso cognitivo): Determina a atração em relação à melhor posição individual
- **c2** (peso social): Determina a atração em relação à melhor posição global

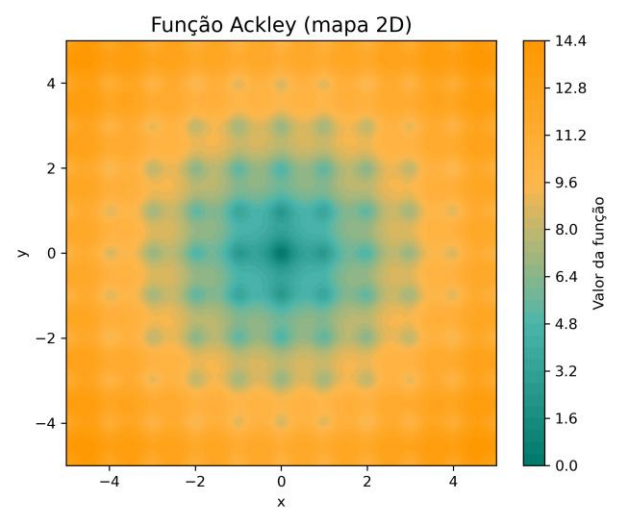
Balanço Exploração/Aproveitamento

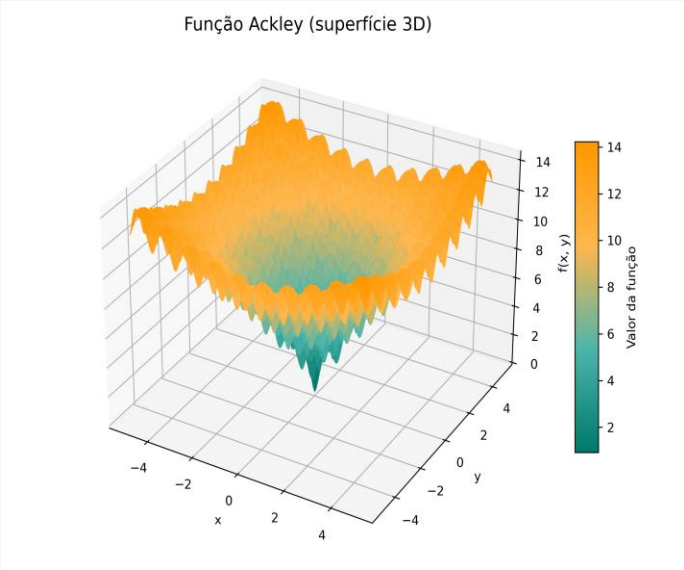
Estes parâmetros são ajustados para equilibrar a exploração e o aproveitamento no espaço de procura, ajudando o algoritmo a encontrar soluções ótimas.

```python
def run_pso(n_agents=4, iters=6, w=0.7, c1=1.7, c2=1.7):
    """
    Executa o PSO com parâmetros clássicos:
      w  = inércia da partícula
      c1 = componente cognitiva (melhor posição individual)
      c2 = componente social   (melhor posição global)
    """
    opt = pso(n_agents, fitness, LB, UB, 2, iters, w, c1, c2)
    return opt.get_Gbest()
```

Excerto de código de **optimize_gsa_pso.py**

Função Ackley





- **Mínimo global conhecido** - Localizado na vizinhança da origem
- **Superfície altamente não linear** com múltiplos mínimos locais
- **Validação** - Permite avaliar a capacidade de exploração e convergência dos algoritmos Swarm
- **Testes** - Realizados em 2D e 3D para diferentes complexidades de espaço

Configuração dos testes (Ackley)

```python
def run_ackley(dim=2, n=30, iters=200, lb=-5.0, ub=5.0, gsa_g0=3.0, pso_params=None, test_name="Padrão"):

    # ------------------------
    # GSA
    # ------------------------
    G = gsa_fn(n, tf.ackley_function, lb, ub, dim, iters)
    g_best_pos = list(map(float, G.get_Gbest()))
    g_best_val = f_ackley(g_best_pos)

    # ------------------------
    # PSO (com variação de W)
    # ------------------------
    pso_config = pso_params if pso_params else {"w": 0.7, "c1": 1.7, "c2": 1.7}

    P = pso_fn(
        n, tf.ackley_function, lb, ub, dim, iters,
        pso_config.get("w", 0.7),
        pso_config.get("c1", 1.7),
        pso_config.get("c2", 1.7)
    )
    p_best_pos = list(map(float, P.get_Gbest()))
    p_best_val = f_ackley(p_best_pos)

    return {
        "Teste": test_name,
        "dim": dim, "n": n, "iters": iters, "lb": lb, "ub": ub,
        "GSA_best_pos": g_best_pos, "GSA_best_val": g_best_val,
        "PSO_best_pos": p_best_pos, "PSO_best_val": p_best_val,
        "PSO_w": pso_config.get("w", 0.7),
        "PSO_c1": pso_config.get("c1", 1.7),
        "PSO_c2": pso_config.get("c2", 1.7),
    }
```

Função
Implementação da função de Ackley

Limites
[-5, 5] para todas as dimensões

Dimensões
Testes em 2D e 3D

Parâmetros
PSO: w=0.7, c_1=c_2=1.7

GSA usa a versão base da **biblioteca**, que não permite ajustar G_0.

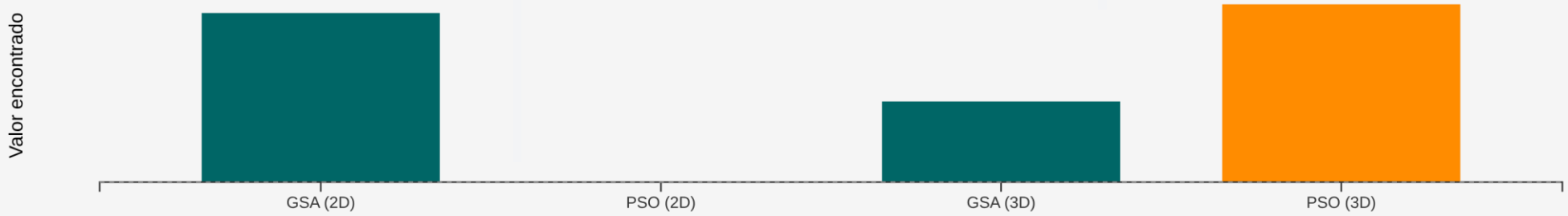
O parâmetro G_0=3 aparece no script mas não tem impacto na execução.

Excerto de código de **ackley_benchmark.py**

Resultados na Função Ackley (2D e 3D)

Comparação do desempenho dos algoritmos GSA e PSO na otimização da função de Ackley em 2D e 3D:

Dimensão	GSA	PSO
2D	0.0035	**6.64×10^{-7}**
3D	**-1.7583**	-1.7080



- **PSO** foi mais preciso em 2D, encontrando um valor muito próximo do mínimo global.
- **GSA** obteve um valor ligeiramente melhor em 3D em comparação com PSO.

Sensibilidade do PSO (inércia W)

Análise do fator de inércia (W) no desempenho do PSO na função de Ackley:

Configuração	W (Inércia)	C_1 C_2 (Aceleração)	PSO Melhor Valor (Fitness)	Resultado
Padrão	0.7	1.7, 1.7	6.64×10^{-7}	
Alta Inércia	1.0	1.7, 1.7	1.39	
Baixa Inércia	0.4	1.7, 1.7	0.0	

Análise

- W=0.7 proporciona boa aproximação ao ótimo global
- W=1.0 resultou em convergência significativamente pior
- W=0.4 convergiu rapidamente para valores muito baixos, levando ao valor mínimo registado (0.0)

Conclusão

Para a função de Ackley, a inércia reduzida (W=0.4) favoreceu uma convergência mais estável, atingindo o valor mínimo observado.
Já a inércia elevada (W=1.0) aumentou a exploração e dificultou a estabilização.

A CNN usada

Entrada
Imagens RGB 64×64×3

5 Blocos Convolucionais

Conv2D
Filtros: 32, 64, 128, 256, 256
Kernel: 3×3, Ativação: ReLU

BatchNorm
Normalização

MaxPooling
Redução dimensional (2×2)

Camada Flatten
Converte para vetor 1D

Camada Dense
128 neurónios, ativação ReLU

Dropout
Taxa variável (hiper-parâmetro)

Camada de Saída
5 neurónios, ativação Softmax

Características

- Arquitetura “avançada"
- 5 blocos convolucionais
- Saída softmax para 5 classes
- Dropout como regularizador

```python
# Função que cria a CNN
def create_cnn_model(input_shape, num_classes, variant="advanced", lr=0.003614, dropout=0.25):
    model = Sequential()
    model.add(Input(shape=input_shape))

    # 1º bloco - padrões básicos (bordas)
    model.add(Conv2D(32, (3,3), activation='relu', padding='same'))
    model.add(BatchNormalization())
    model.add(MaxPooling2D((2,2)))

    # ...

    # 5º bloco - reforço de features profundas
    model.add(Conv2D(256, (3,3), activation='relu', padding='same'))
    model.add(BatchNormalization())
    model.add(MaxPooling2D((2,2)))

    # Parte final da rede
    model.add(Flatten())
    model.add(Dense(128, activation='relu'))
    model.add(Dropout(dropout))
    model.add(Dense(num_classes, activation='softmax'))
    return model
```

</> Excerto de código de **cnn_model.py**

Função de Fitness

Objetivo

Otimização dos hiper-parâmetros **lr** (learning rate) e **dropout** da CNN

Implementação

A função de fitness é definida no ficheiro **optimize_gsa_pso.py**

Características

- **Garantia de validade**: clipping automático + penalização forte (1000) para valores proibidos.

  Fitness = -val_accuracy

- 3 épocas por avaliação
- As constantes **INPUT_SHAPE** e **NUM_CLASSES** estão definidas no início do mesmo ficheiro.

⚠️ **Importante: Os algoritmos swarm são de minimização!**

```python
def fitness(params):
    # Converter para numpy array
    p = np.asarray(params, dtype=float)

    # Penalizar casos inválidos
    if np.any(np.isnan(p)) or np.any(np.isinf(p)):
        return 1e3

    # Respeitar os limites definidos [LB, UB]
    p = np.clip(p, LB, UB)
    lr, dr = float(p[0]), float(p[1])

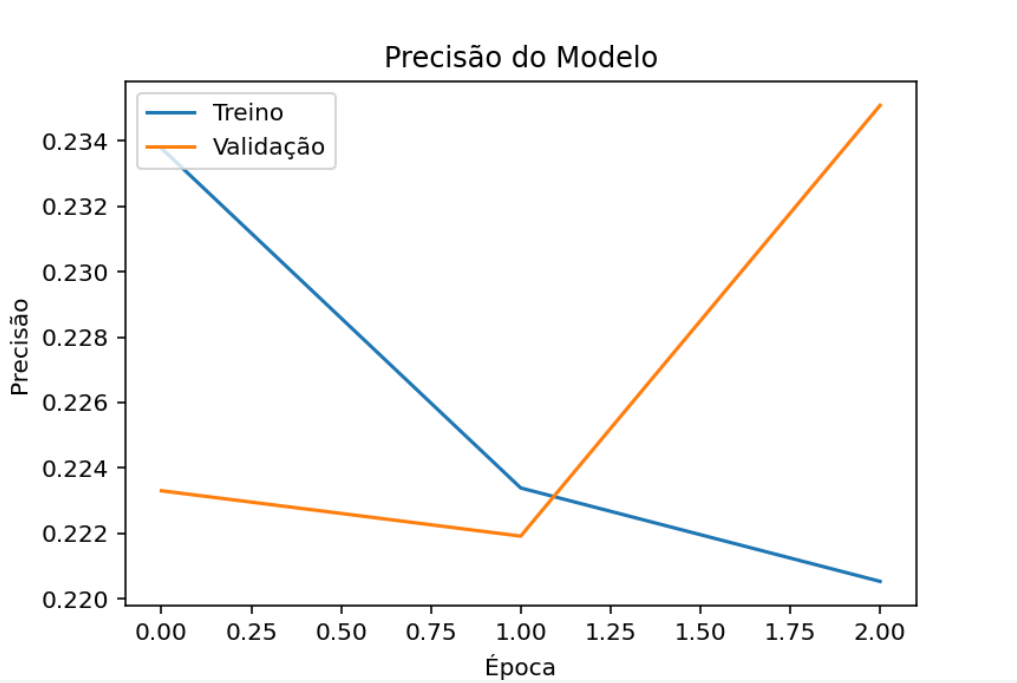
    # Segurança extra (dropout nunca deve ultrapassar 0.9)
    if not (1e-6 < lr <= 1.0) or not (0.0 <= dr <= 0.9):
        return 1e3

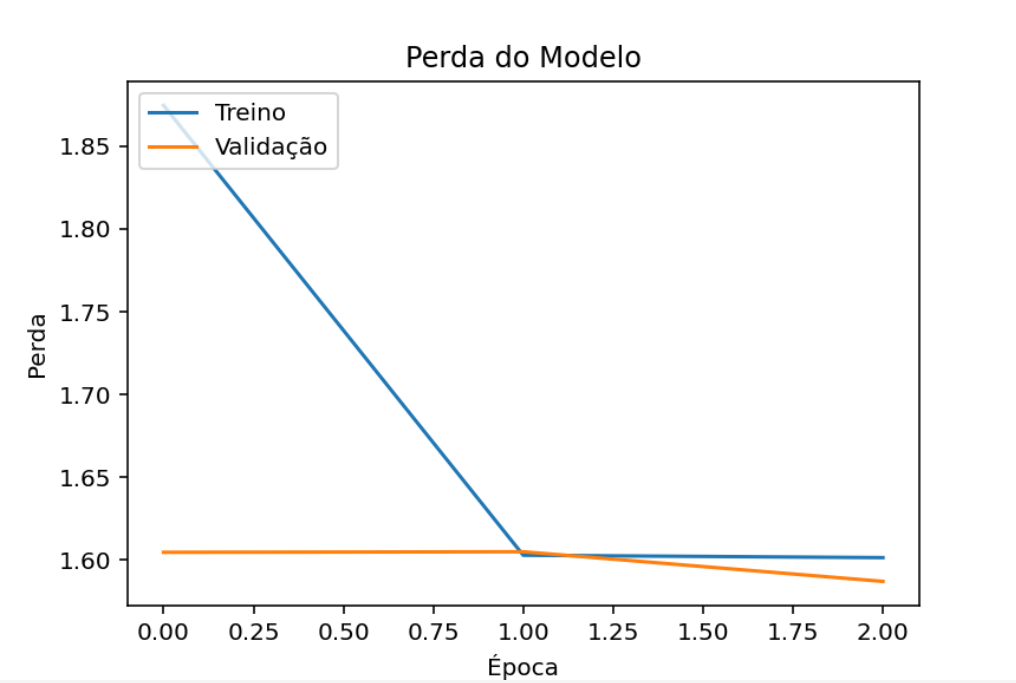
    # Treino real da CNN
    # epochs = 3
    val_acc, _ = train_model(
        TRAIN_DIR,
        VAL_DIR,
        input_shape=INPUT_SHAPE,
        num_classes=NUM_CLASSES,
        batch_size=64,
        epochs=3,          # cada avaliação do fitness treina 3 épocas
        img_size=64,
        lr=lr,
        dropout=dr,
        variant="advanced",
    )

    # Penalização se accuracy for inválida
    if math.isnan(val_acc) or math.isinf(val_acc):
        return 1e3

    # Swarm minimiza → devolver negativo
    return -float(val_acc)
```

Treino da CNN (gráficos)





Resultados

A precisão de treino desce, enquanto a validação melhora na última época.
A perda de treino reduz bastante e a perda de validação mantém-se estável, com ligeira melhoria no final.

Os gráficos apresentados correspondem ao PSO, que foi o algoritmo com melhor desempenho.
Como o GSA apresentou valores de validação significativamente inferiores, não justificava a apresentação dos seus gráficos.

Resultados finais (CNN)

Algoritmo	Learning rate	Dropout
GSA	0.0032	0.12
PSO	0.00877	0.4680

Insights Finais

Os hiper-parâmetros encontrados pelo **PSO** resultaram numa **val_accuracy** perto de **0.52**

Os hiper-parâmetros encontrados pelo **GSA** produziram uma **val_accuracy** muito inferior, cerca de **0.22**

Isto mostra que a escolha do algoritmo tem impacto direto no desempenho final da CNN

Conclusões

Os algoritmos de Swarm mostraram-se **eficazes** na otimização de hiper-parâmetros, especialmente em funções com muitos mínimos locais, como a Ackley.

O PSO revelou-se **mais robusto** e encontrou hiper-parâmetros que levaram a uma val_accuracy perto de **0.52**, superando claramente o GSA.

Este trabalho demonstra uma **aplicação prática** bem-sucedida de métodos bio-inspirados na otimização de modelos de Machine Learning, validando a utilidade da Computação Swarm em problemas reais.

